A CAPSTONE PROJECT REPORT

RANSOMWARE DETECTION AND PREVENTION SYSTEM

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA5115 – CRYPTOGRAPHY AND NETWORK SECURITY

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

D. Gnana Deepthi (192411123)

Under the Supervision of

Dr. T. Sangeetha

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Ransomware Detection and Prevention System" has been carried out by D. Gnana Deepthi (192411123) the supervision of Dr, T. Sangeetha and is submitted in partial fulfilment of the requirements for the current semester of the B. E Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. T. Sangeetha
Professor
Computational Intelligence
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. T. Sangeetha
Professor
Computational Intelligence
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I, D. Gnana Deepthi (192411123) of the Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Ransomware Detection and Prevention System" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
D. Gnana Deepthi (192411123)	Module 3 and integrated performance analysis	Emerging storage comparison; storage performance model	Storage metrics, comparison and result interpretation	Module 3, testing, results and reflection	34%
S. Surya shree (192511218)	Module 1 and system architecture	Cache hierarchy and cache-performance model	Cache hit/miss and access-time analysis	Module 1 and system architecture	33%
Shruthi K. (192571019)	Module 2 and integration support	Virtual memory and page-replacement model	Page-hit, page-fault and fault-rate analysis	Module 2 and supporting sections	33%
Total					**100%**

ABSTRACT

Ransomware is a major cybersecurity threat in which malicious software attempts to deny access to information, commonly by modifying or encrypting large numbers of files. A major engineering challenge is detecting the harmful activity quickly enough to reduce the number of files affected. The proposed Ransomware Detection and Prevention System addresses this challenge through a layered architecture in which File Activity Monitoring acts as the primary evidence-collection module.

The File Activity Monitoring module continuously observes relevant file-system activity and records operations such as file creation, modification, writing, renaming, movement and deletion. Rather than repeatedly scanning the entire storage device, the design uses event-driven monitoring so that changes can be captured as they occur. Each observed event is normalized into a structured record containing information such as timestamp, event type, file path, process details where available, file extension and directory context.
Filtering and aggregation are applied to reduce unnecessary noise and convert high-volume low-level events into useful behavioural observations.

The resulting event stream is passed to the Encryption Behaviour Analysis module, where patterns such as rapid modification of many distinct files, repeated renaming, broad directory traversal and abnormal activity rates can be analysed. File Activity Monitoring itself does not make the final ransomware decision; instead, it provides timely and trustworthy evidence to the behavioural-analysis and risk-scoring layers. This separation makes the system modular, explainable and easier to test.

The module is intended to support early warning, evidence logging and preventive response in an authorized environment. Evaluation should measure event-capture reliability, detection latency at the monitoring layer, dropped events under burst load, CPU and memory overhead, filtering effectiveness and the quality of data supplied to downstream analysis. The expected outcome is a practical monitoring component that forms a dependable foundation for ransomware detection and prevention.

Keywords: Ransomware Detection, File Activity Monitoring, File-System Events, Behavioural Analysis, Malware Prevention, Security Monitoring

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	
2	CHAPTER 2: Literature Review	
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	
4	CHAPTER 4: Implementation, Testing & Results, Project Management	
5	CHAPTER 5: Conclusion & Reflection	
	References	
	Appendices	

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	System Implementation	30
2	Dashboard	31

LIST OF TABLES

Table No.	Table Name	Page No.
1	Review of Existing Work	13
2	Comparitive Analysis	13
3	Stakeholder / User Requirements	15
4	Functional and Non Functional Requirements	15
5	Constraints & Applicable Standards	15
6	Constraints & Applicable Standards	16
7	Evaluation	16
8	Trade Off Analysis	17
9	Sustainability, Ethics, Safety, Risk & Security	17
10	Risk Register	18
11	Resource	19
12	Tools Used	19
13	Verification	19
14	Results	20
15	Comparision	22
16	Project Planning	22

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Ransomware attacks can cause severe operational and financial impact because they may modify a large volume of user and organizational data within a short period. From an engineering perspective, protection requires more than recognising a malicious file: the system must also monitor what running processes actually do. File operations provide one of the most direct observable traces of a ransomware attack because encryption-based attacks typically require repeated reads and writes, file replacements, renames and directory traversal. The File Activity Monitoring module is therefore designed as the collection layer of the proposed system. It observes the operating system's file activity, captures relevant events and converts them into structured records that can be analysed by higher-level modules. The design emphasises timely monitoring, reduced noise, secure logging and clear interfaces between components.

1.2 Need for the Project

- Ransomware can modify many files before a user notices the attack.
- Known-malware signatures may not identify previously unseen ransomware variants.
- The detection system requires low-level evidence from file operations before behavioural scoring can occur.
- Continuous monitoring can provide earlier warning than periodic manual inspection.
- Security analysts require event evidence to understand which files, processes and directories

1.3 Problem Statement

Design and implement a File Activity Monitoring module for a ransomware detection and prevention system that can continuously observe relevant file-system events, capture sufficient process and file context where available, reduce monitoring noise, aggregate activity over configurable time windows, and provide timely structured evidence to the Encryption Behaviour Analysis and prevention modules while maintaining acceptable system overhead.

1.4 Project Aim

To develop a reliable File Activity Monitoring module that provides real-time or near-real-time file-

system evidence for behavioural ransomware detection and prevention.

1.5 Project Objectives

- To design an event-driven file activity collection mechanism.
- To capture creation, modification, write, rename and deletion activity.
- To associate events with process information where supported by the operating system.
- To normalize and filter events into a common data structure.
- To aggregate events to calculate useful activity rates and distinct-object counts.
- To transfer structured observations to Encryption Behaviour Analysis.
- To evaluate capture reliability, latency and monitoring overhead.

1.6 Scope

Included: monitoring of selected directories or protected locations; event collection; normalization; filtering; time-window aggregation; process context where available; logging; integration with behavioural analysis; and performance evaluation. Excluded: recovery of encrypted files, offensive ransomware creation, uncontrolled malware execution, and deployment on production systems without authorization. The module is assumed to operate within a controlled and approved test environment.

1.7 Expected Engineering Outcome

The expected outcome is a working software module or prototype that can observe and record relevant file activity, produce structured events for behavioural analysis, and demonstrate stable performance under normal and burst workloads. The module should provide a traceable relationship between observed file operations and the downstream ransomware risk decision

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review should consider research on ransomware detection, behavioural malware analysis, file-system monitoring, anomaly detection, endpoint security and automated response. Existing security products and operating-system event-monitoring mechanisms should be compared according to their visibility, latency, implementation complexity, resource overhead and ability to support downstream behavioural analysis.

2.2 Review of Existing Work

Authors	Title / Focus	Work Done	Drawback
Sethumurugan et al. (2021)	**Cost-effective cache replacement using ML**	**Lightweight predictor for shared last-level cache replacement**	**Needs workload retraining and adds hardware overhead**
Yang et al. (2023)	**PAIC cache replacement**	**Adapted replacement decisions to prefetch-demand interactions**	**Extra predictor storage and benchmark-dependent gains**
Kumar et al. (2021)	**Radiant page-table management**	**Page-table placement for tiered memory**	**Adds OS-level complexity and hardware assumptions**
Skarlatos et al. (2023)	**Contiguitas**	**Contiguity-aware allocation to reduce translation overhead**	**Requires allocator/OS changes**
Xiang et al. (2022)	**Intel Optane buffering**	**Latency and bandwidth characterization**	**Hardware generation studied is discontinued**

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Signature-based detection	Fast for known threats	Limited visibility of novel behaviour	Provides baseline security layer

Periodic directory scanning	Simple to implement	Delayed detection; higher repeated I/O	Useful for comparison only
Event-driven monitoring	Immediate activity visibility	Requires event handling and filtering	Primary monitoring design
Machine-learning anomaly detection	Can combine complex features	Needs data and tuning	Future enhancement to EBA
Proposed modular FAM	Timely, structured, explainable evidence	Requires platform-specific integration	Selected design

2.4 Research / Engineering Gap

The main engineering gap is not simply obtaining file events; it is transforming high-volume operating-system activity into a reliable, low-latency data stream that is useful for ransomware analysis. A practical capstone solution should be modular, testable and explainable. The proposed File Activity Monitoring module addresses this gap through event normalization, noise filtering, aggregation and a defined interface with Encryption Behaviour Analysis.

2.5 Proposed Contribution

The contribution of this module is a structured collection and processing layer designed specifically for ransomware-oriented analysis. It preserves important context, reduces irrelevant activity and exposes aggregated measures that can be directly consumed by the behavioural analysis and risk-scoring stages.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	**Requirement**
End User	Monitor and protect important files without noticeable disruption.
System Administrator	Configure monitored paths and review alerts/logs.
Security Analyst	Obtain event evidence with time, operation, path and process context where available.
Project Evaluator	Verify the module using measurable performance and reliability tests.

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Capture file events	Required event types captured during test scenarios
Functional	Normalize events	Common event schema used by all records
Functional	Filter activity	Configured noise and exclusions handled correctly
Functional	Aggregate events	Time-window counts and distinct-object statistics calculated
Functional	Forward data	Structured events delivered to EBA
Functional	Log evidence	Event records available for verification
Non-Functional	Low overhead	Within project-defined CPU/RAM limit
Non-Functional	Low latency	Within project-defined delay
Non-Functional	Reliability	Minimal event loss during normal load
Non-Functional	Security	Restricted modification of logs/configuration

3.2 Constraints & Applicable Standards

The monitoring module must be developed and tested only within an authorized environment. Since the project involves cybersecurity behaviour, testing should use synthetic data and safe simulations. Technical constraints include operating-system differences in event APIs, process-attribution availability, event burst rates and storage overhead from logging. Institutional cybersecurity, software engineering and data-protection requirements applicable to the project should be documented in the final submission.

Constraint / Requirement	How Applied in This Project
Controlled testing	Use isolated VM/test environment and synthetic files.

Data minimization	Collect only technical metadata needed for detection.
Access control	Restrict modification of logs and configuration.
Platform compatibility	Document the target operating system and API used.
Performance	Measure CPU, RAM, latency and event throughput.

3.3 Design Specifications

Parameter	**Target / Requirement**	**Priority**
Monitoring mode	Event-driven / continuous	High
Monitored paths	Configurable	High
Observation window	Configurable time interval	High
Event schema	Standardized fields	High
Process attribution	Where OS support exists	Medium
Logging	Structured and reviewable	High
CPU/RAM overhead	Low / project-defined	Medium
Queue capacity	Sufficient for burst events	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Periodic scanning compares directory state at fixed intervals. It is simple but can miss short-lived activity and introduces repeated storage reads. Alternative 2: Event-driven monitoring receives file-system notifications as they occur. It provides faster visibility but needs platform-specific event handling and noise filtering. Alternative 3: Hybrid monitoring combines event-driven collection with occasional validation scans. It can improve resilience to missed events but adds complexity and resource usage.

Criterion	**Weight**	**Periodic Scan**	**Event-Driven**	**Hybrid**
Performance	25	3	5	4
Detection Timeliness	25	2	5	4
Implementation Complexity	15	4	3	2
Scalability	15	3	5	4
Reliability	10	3	4	5

Maintainability	10	4	4	3

3.5 Selected Design & Detailed Design

Event-driven monitoring is selected because detection timeliness is critical for ransomware. The module is designed as a pipeline that receives file-system notifications, normalizes each event, applies scope and noise rules, updates an observation window, writes required audit information and forwards structured data to Encryption Behaviour Analysis.

Component	**Responsibility**
Scope Manager	Defines monitored paths, exclusions and policy.
Event Listener	Receives file-system notifications.
Event Normalizer	Converts raw notifications to the common schema.
Process Mapper	Associates events with a process where supported.
Noise Filter	Removes or reduces irrelevant activity.
Event Aggregator	Maintains counts, rates and unique-object statistics.
Event Queue	Provides buffered handoff to analysis.
Logger	Stores technical evidence securely.
EBA Interface	Passes structured observations downstream.

3.7 Trade-off Analysis

Design Decision	**Alternative Considered**	**Benefit**	**Disadvantage**	**Final Decision & Justification**
Event-driven collection	Periodic scanning	Low latency	Platform-dependent event handling	Selected because early detection is the primary objective
Multi-stage filtering	Ignore all high-volume sources	Reduces noise while retaining context	More configuration required	Selected to balance visibility and overhead
Buffered queue	Direct synchronous processing	Protects monitoring responsiveness during bursts	Queue sizing required	Selected for burst handling

Structured logging	Free-form text logging	Easier downstream parsing and analysis	More design work	Selected for reliable evaluation and integration

3.8 Sustainability, Ethics, Safety, Risk & Security

Domain	**Consideration**	**Evidence Evaluated**	**Impact on Final Decision**
Sustainability	Avoid unnecessary continuous scanning	CPU/I/O measurements	Event-driven monitoring selected
Ethics	Authorized security testing	Project/lab authorization and test plan	Synthetic and isolated tests required
Health & Safety	Prevent unintended system damage	Isolation and rollback procedure	No uncontrolled malware execution
Security	Protect monitoring evidence	Access and integrity controls	Restricted logs/configuration

Risk Register

Risk	**Likelihood**	**Severity**	**Risk Level**	**Mitigation**	**Residual Risk**
Event loss during bursts	Medium	High	High	Buffered queue, aggregation and load testing	Medium
False positives from legitimate bulk activity	Medium	High	High	Multi-feature downstream analysis and filtering	Medium
Monitoring overhead	Medium	Medium	Medium	Event-driven collection and efficient logging	Low

Incorrect process mapping	Medium	Medium	Medium	Validate against platform-specific tests	Low
Log tampering	Low	High	Medium	Restrict access and monitor integrity	Low
Unsafe testing	Low	High	High	Isolated environment and synthetic data	Low

CHAPTER 4
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The implementation follows an incremental development approach. First, file events are captured from benign test activity. The event schema and filtering rules are then validated. Next, aggregation and queueing are integrated, followed by the interface to Encryption Behaviour Analysis and secure logging. Finally, controlled ransomware-like file-activity simulations using synthetic files are used to evaluate event capture, latency, reliability and system overhead.

Resource	**Purpose**
Development environment	Implementation and debugging
Target operating system APIs	File-system event capture
Synthetic test dataset	Repeatable monitoring scenarios
Structured log format	Event storage and analysis
Virtual machine / sandbox	Safe controlled testing
Plotting / analysis tool	Performance evaluation

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

The monitoring service starts by loading and validating configuration. It then registers monitored locations with the target operating-system event interface. When an event arrives, the module validates the event, determines the operation type, normalizes the timestamp and path, retrieves process information where available, applies filtering rules, updates aggregation counters, writes required logging information and forwards the structured record or aggregate to the behavioural analysis interface. To preserve responsiveness, event capture should be separated from expensive processing.

Tool / Software / Equipment	**Purpose**	**Stage Used**
Programming language selected for prototype	Event monitoring and data processing	Development
Operating-system file event interface	Receive notifications	Implementation
CSV / JSON or equivalent structured logging	Store and review events	Testing

Spreadsheet / plotting environment	Graph metrics	Analysis
Virtual machine or isolated host	Safe validation	Testing

4.3 Test Plan & Verification

Requirement	**Test Method**	**Acceptance Criterion**
File creation capture	Create controlled files	CREATE events recorded
File modification capture	Edit/write synthetic documents	WRITE/MODIFY events recorded
Rename capture	Rename test files	RENAME/MOVE events recorded
Delete capture	Delete synthetic files	DELETE events recorded where supported
Normal workload	Run common benign file operations	No monitoring failure and no excessive overhead
Burst workload	Generate high-rate synthetic events	Monitoring remains responsive and event loss is within target
Process context	Use controlled application process	Process information captured where OS supports mapping
Filtering	Generate included and excluded events	Policy applied correctly
Logging	Inspect event records	Required fields are present
Latency	Compare event occurrence and processing timestamps	Within project-defined latency target

4.4 Results

Requirement	**Test Method**	**Acceptance Criterion**
File creation capture	Create controlled files	CREATE events recorded
File modification capture	Edit/write synthetic documents	WRITE/MODIFY events recorded
Rename capture	Rename test files	RENAME/MOVE events recorded
Delete capture	Delete synthetic files	DELETE events recorded where supported

Normal workload	Run common benign file operations	No monitoring failure and no excessive overhead
Burst workload	Generate high-rate synthetic events	Monitoring remains responsive and event loss is within target
Process context	Use controlled application process	Process information captured where OS supports mapping
Filtering	Generate included and excluded events	Policy applied correctly
Logging	Inspect event records	Required fields are present
Latency	Compare event occurrence and processing timestamps	Within project-defined latency target

Metric	**Required / Target**	**Achieved Value**	**Status**
Event capture reliability	Project-defined target	To be measured	Pending
Processing latency	Project-defined target	To be measured	Pending
Event loss under burst	Project-defined maximum	To be measured	Pending
CPU overhead	Project-defined maximum	To be measured	Pending
Memory overhead	Project-defined maximum	To be measured	Pending

4.5 Data Analysis & Engineering Judgment

Monitoring performance should be evaluated under both normal and burst conditions. Capture reliability is important because missing a short sequence of file events could reduce the effectiveness of downstream behavioural analysis. CPU and memory usage should be compared with a baseline system that runs without the monitoring service. Latency should be measured from the event timestamp to the time at which a structured event becomes available to the analysis layer.

Engineering judgment should focus on balancing complete visibility against system overhead. When event volumes are high, aggregation can reduce downstream load without losing the information required for ransomware detection. However, aggregation must preserve distinct-object counts and timing information so that rapid mass modifications remain visible.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design event-driven file monitoring	Working event listener and configured paths	To be evaluated
Capture required event types	Test-case logs	To be evaluated
Normalize and filter events	Structured records and filter tests	To be evaluated
Aggregate activity	Observation-window metrics	To be evaluated
Integrate with EBA	Structured interface/output	To be evaluated
Evaluate performance	CPU/RAM/latency/load measurements	To be evaluated

4.7 Strengths & Limitations

Strengths:

- Provides timely low-level evidence for ransomware analysis.
- Separates collection from final malware classification.
- Supports filtering and aggregation to control event volume.
- Produces explainable technical records.
- Can be extended with additional process and storage indicators.

Limitations:

- Event interfaces vary by operating system.
- Some systems may not expose complete process attribution for every event.
- High event bursts may challenge queues and logging.
- Legitimate bulk file activity can resemble malicious behaviour.
- Monitoring alone cannot determine malicious intent.
- The module does not guarantee detection of every ransomware technique.

4.8 Project Planning, Roles & Budget

Milestone	Weeks	Deliverable
Problem definition and review	1–2	Requirements and literature review
Architecture and event schema	3–4	Monitoring design
Event collection	5–6	Working listener
Filtering and aggregation	7–8	Processed event stream

Integration and testing	9–10	Connected EBA interface and tests
Analysis and documentation	11–12	Final results and report
Student	**Assigned Responsibility**	**Actual Contribution**
G.Gnana Deepthi (192411123)	File Activity Monitoring module, testing, analysis and documentation	100%
Item	**Estimated Cost**	**Actual Cost**
Development software	₹0–₹5,000	To be entered
Test VM / storage resources	₹0–₹3,000	To be entered
Documentation / miscellaneous	₹0–₹1,000	To be entered
Total	₹0–₹9,000	To be entered

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The File Activity Monitoring module provides the foundational evidence-collection capability for the Ransomware Detection and Prevention System. By continuously observing file-system activity, normalizing events, filtering irrelevant noise and aggregating related operations, the module creates a structured stream that can be analysed for ransomware-like behaviour. This architecture is particularly useful because file modification and renaming activity are direct observable consequences of encryption-oriented attacks. The selected event-driven design prioritizes detection timeliness while maintaining modularity. The interface to Encryption Behaviour Analysis allows the monitoring component to focus on reliable event collection

5.2 Future Work

- Add stronger process provenance and parent-child process tracking.
- Introduce adaptive baselines for normal file activity.
- Integrate optional machine-learning anomaly detection.
- Use protected snapshots or backup verification as a prevention mechanism.
- Add central dashboards for event-rate and risk-score visualization.
- Improve cross-platform support for Windows and Linux event interfaces.
- Correlate file activity with registry, memory and network indicators where appropriate.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- File-system event monitoring and event-driven programming.
- Behavioural indicators used in ransomware detection.
- Event filtering, aggregation and queue design.
- Performance measurement and security-test

methodology. Engineering & Professional Skills

Developed:

- System architecture and modular design.
- Experimental planning and measurable verification.
- Technical documentation and engineering communication.

- Risk assessment, ethical testing and evidence-based design.

Individual Reflection – G.Gnana Deepthi (192411123): The main engineering challenge was determining how to monitor high-volume file activity without treating every normal file operation as malicious. I addressed this by designing a structured event pipeline with filtering, aggregation and a clear interface to behavioural analysis. A key decision was to use event-driven monitoring because ransomware can modify many files rapidly and delayed periodic scanning may reduce detection timeliness. I also learned the importance of validating performance and false-positive conditions rather than assuming that a monitoring service is effective simply because it captures events. If the project were repeated, I would collect more diverse benign workloads and platform-specific event traces before final threshold calibration.

REFERENCES

The final submission should use a consistent recognised referencing style such as IEEE. The reference list should include every research paper, standard, website, dataset, software package, open-source resource and AI-assisted resource actually used in the project.

- Research literature on ransomware behavioural detection and file-system monitoring.
- NIST cybersecurity and incident-response guidance relevant to malware detection and response.
- MITRE ATT&CK resources relevant to ransomware behaviour and file/directory modification.
- Official operating-system documentation for the file-system event API used in the prototype.
  Documentation for programming libraries, logging tools and analysis software used in implementation

APPENDICES

APPENDIX A – SYSTEM MODULE DESCRIPTION

D.1 Module 1 – File Activity Monitoring

The File Activity Monitoring module continuously observes file operations performed on the system. It tracks file creation, modification, deletion, and renaming activities. The frequency and pattern of these operations are analyzed to identify unusual behavior.

Rapid modifications or changes across a large number of files are treated as suspicious activity. The detected events are forwarded to the threat detection stage for further analysis. This module helps in the early identification of ransomware-like behavior. TEAM 5 PPT.pptx

D.2 Module 2 – Malware Signature Detection

The Malware Signature Detection module identifies known ransomware using predefined malware signatures. Suspicious files or processes are analyzed and their characteristics are compared with the available signature database.

If a matching signature is found, the file or process is classified as a known malware threat and forwarded to the prevention mechanism. This provides fast detection of previously identified ransomware.

D.3 Module 3 – Encryption Behaviour Analysis

The Encryption Behaviour Analysis module identifies abnormal encryption patterns. It monitors rapid encryption and large-scale modifications occurring across multiple files within a short period.

APPENDIX E – ALGORITHM

E.1 Ransomware Detection Algorithm

Step 1: Start the system.

Step 2: Monitor file activities such as creation, modification, deletion, and renaming.

Step 3: Collect recent file activity events.

Step 4: Calculate the SHA-256 hash of suspicious files.

Step 5: Compare the calculated hash with the malware signature database.

Step 6: If a matching signature is found, classify the file as a known malware threat.

Step 7: Analyze the frequency of file modifications and encryption-related activities.

Step 8: Calculate the threat score based on abnormal behavior.

Step 9: If the threat score reaches the critical level, identify the activity as ransomware-like behavior.

Step 10: Trigger the threat prevention mechanism.

Step 11: Restrict further malicious file operations.

Step 12: Generate a real-time security alert.

Step 13: Continue monitoring the system.

Step 14: Stop.

APPENDIX F – SOFTWARE IMPLEMENTATION

F.1 SHA-256 Hash Calculation

The system uses SHA-256 hashing to generate a unique hash value for a file. The generated hash is used for signature matching.

```
def calculate_hash(file_path):
    sha256 = hashlib.sha256()
    with open(file_path, "rb") as file:
        while data := file.read(4096):
            sha256.update(data)
    return sha256.hexdigest()
```

F.2 Signature Matching

The calculated file hash is compared with the stored malware signatures.

```
def check_signature(file_path):
    file_hash = calculate_hash(file_path)
    cursor.execute(
        "SELECT threat_name FROM signatures WHERE file_hash = ?",
        (file_hash,)
    )
    return cursor.fetchone()
```

APPENDIX G – BEHAVIOUR ANALYSIS AND THREAT PREVENTION

G.1 Behaviour Analysis

The behavior analysis mechanism assigns a threat score according to suspicious file activity.

```
def analyze_behavior(recent_events):
    score = 0
    if len(recent_events) >= 10:
        score += 30
```

```
    if monitor.modified >= 10:
        score += 20
    if monitor.created >= 5:
        score += 10
    if monitor.renamed >= 5:
        score += 10
    return min(score, 100)
```

G.2 Threat Prevention

When the calculated threat score reaches the critical threshold, the system generates an alert and activates the prevention mechanism.

```
if score >= 70:
    threat_label.config(
        text="Threat Level: CRITICAL"
    )
    trigger_prevention(
        "Ransomware-like activity",
        score
    )
```

APPENDIX H – TEST CASES AND EXPECTED RESULTS

Test Case	Input / Activity	Expected Result
TC01	Normal file creation	File activity recorded without threat alert
TC02	Normal file modification	Activity monitored normally
TC03	Multiple rapid file modifications	Activity identified as suspicious
TC04	Known malicious file	Signature matching identifies the threat
TC05	Unknown suspicious file activity	Behavioral analysis evaluates the activity
TC06	Multiple files encrypted rapidly	Ransomware-like behavior detected
TC07	Threat score ≥ 70	Critical threat alert generated
TC08	Detected ransomware activity	Prevention mechanism triggered
TC09	Normal system activity	No ransomware alert generated
TC10	Suspicious file operations	Real-time security notification displayed

H.1 Expected System Output

The system is expected to:

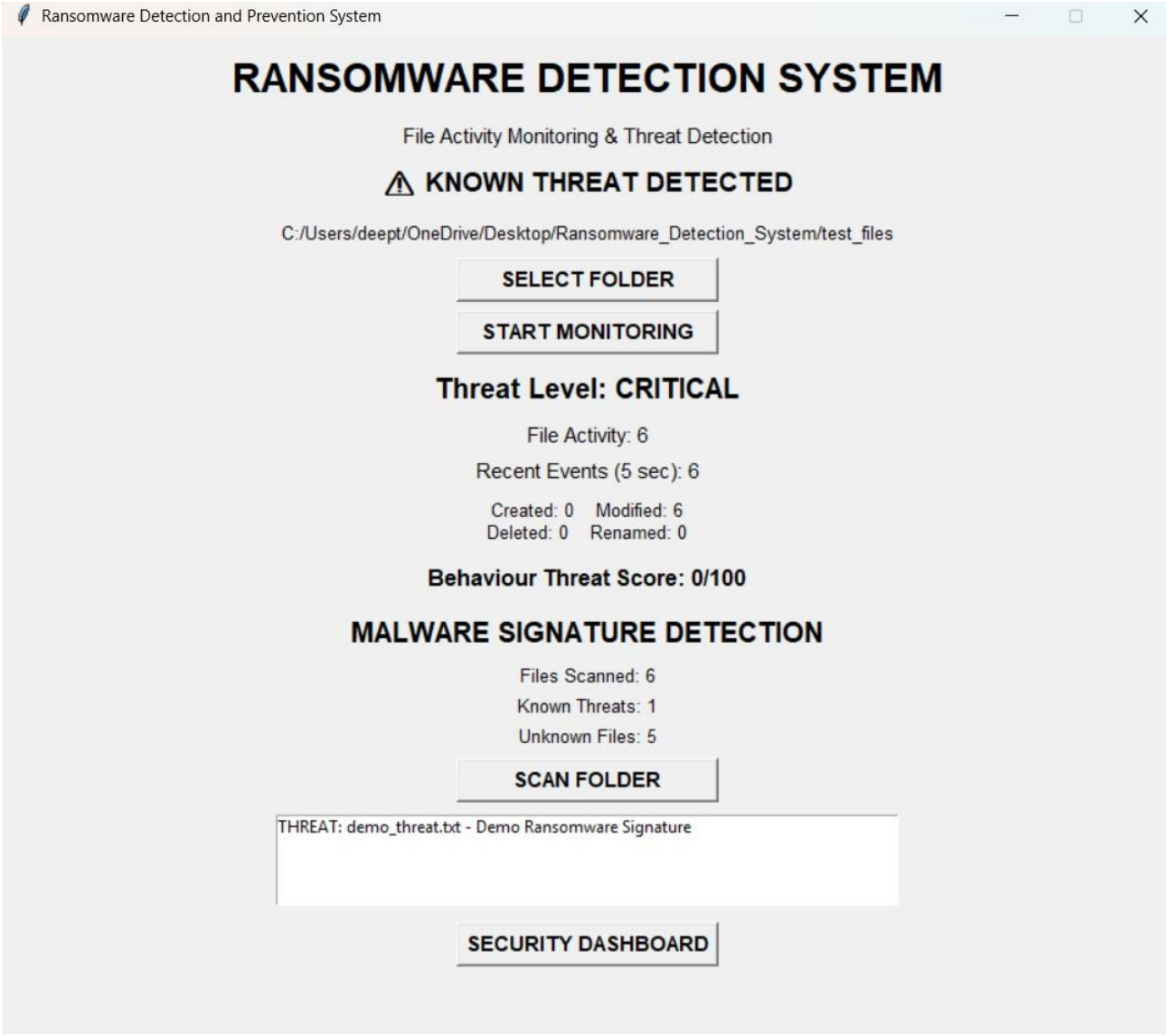
* Monitor file activities continuously.
* Detect known ransomware using signatures.
* Identify abnormal encryption behavior.
* Generate real-time alerts.
* Restrict further malicious file operations.
* Reduce potential file damage and data loss.

These expected outcomes are consistent with the reported project results. TEAM 5 PPT.pptx

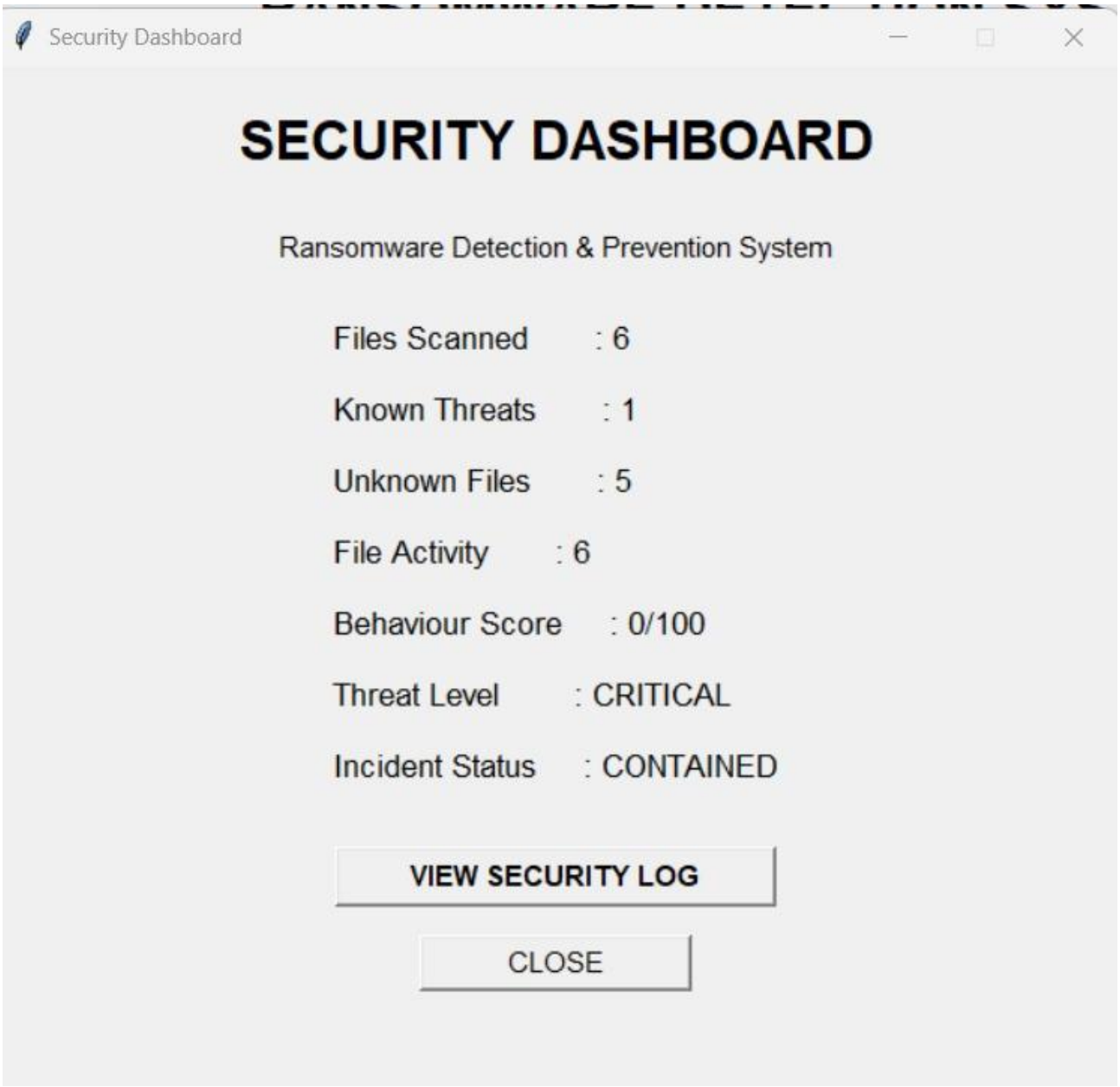
APPENDIX I – SYSTEM OUTPUT / SCREENSHOTS

I.1 Dashboard Functions

The security dashboard provides information regarding the security status of the system. It can be used to display detected suspicious activities, threat levels, and security alerts.



Detection system Implementation



DASHBOARD

APPENDIX J – APPLICATIONS AND FUTURE ENHANCEMENTS

J.1 Applications

The proposed system can be used in the following areas:

1. Protection of computers against malware and ransomware attacks.
2. Real-time monitoring of suspicious file activities.
3. Prevention of unauthorized file encryption and data loss.
4. Generation of real-time security alerts.
5. Isolation of affected files or processes.
6. Assistance to system administrators in identifying security threats.
7. Supporting organizations in maintaining system and data security.
8. Faster identification and response to ransomware attacks. TEAM 5 PPT.pptx

J.2 Future Enhancements

The system can be enhanced in the following ways:

* Integration of Machine Learning for detecting unknown ransomware variants.
* Advanced behavioral analysis for improving detection accuracy.
* Integration with cloud-based threat intelligence for updated malware signatures.
* Automated file backup and recovery mechanisms.
* Extension of the system for enterprise-level ransomware protection.
* Improved real-time monitoring and reporting.
* Faster automated threat response and prevention. TEAM 5 PPT.pptx

J.3 Overall Appendix Summary

The appendices provide the detailed technical information supporting the Ransomware Detection and Prevention System, including module descriptions, algorithm, software implementation, behavior analysis, test cases, system outputs, applications, and future enhancements. The project follows a multi-layered detection approach combining file activity monitoring, malware signature detection, encryption behavior analysis, threat prevention, and real-time alerts. TEAM 5 PPT.pptx

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)